

```

1 //#####
2 // FILE:   HWstarter_main.c
3 //
4 // TITLE:   HW Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "driverlib.h"
16 #include "device.h"
17 #include "F28379dSerial.h"
18 #include "LEDPatterns.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22
23 #define PI          3.1415926535897932384626433832795
24 #define TWOPI       6.283185307179586476925286766559
25 #define HALFPI      1.5707963267948966192313216916398
26 // The Launchpad's CPU Frequency set to 200 you should not change this value
27 #define LAUNCHPAD_CPU_FREQUENCY 200
28
29
30 // Interrupt Service Routines predefinition
31 __interrupt void cpu_timer0_isr(void);
32 __interrupt void cpu_timer1_isr(void);
33 __interrupt void cpu_timer2_isr(void);
34 __interrupt void SWI_isr(void);
35
36 float saturate(float, float);
37
38 // Count variables
39 uint32_t numTimer0calls = 0;
40 uint32_t numSWIcalls = 0;
41 extern uint32_t numRXA;
42 uint16_t UARTPrint = 0;
43 uint16_t LEDdisplaynum = 0;
44
45
46 // Exercise 8 global variables
47 float sinvalue = 0;
48 float time = 0;
49 float ampl = 3.0;
50 float frequency = 0.05;
51 float offset = 0.25;
52 int32_t timeint = 0;
53 float satvalue = 0;
54 #define SAT_LIMIT 2.65
55
56 void main(void)
57 {
58     // PLL, WatchDog, enable Peripheral Clocks
59     // This example function is found in the F2837xD_SysCtrl.c file.
60     InitSysCtrl();
61
62     InitGpio();
63
64     // Blue LED on LaunchPad
65     GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
66     GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
67     GpioDataRegs.GPASET.bit.GPIO31 = 1;
68
69     // Red LED on LaunchPad
70     GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
71     GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
72     GpioDataRegs.GPBSET.bit.GPIO34 = 1;
73
74     // LED1 and PWM Pin
75     GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 0);
76     GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);
77     GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
78
79     // LED2
80     GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
81     GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
82     GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
83
84     // LED3
85     GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
86     GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
87     GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
88
89     // LED4
90     GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
91     GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
92     GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
93
94     // LED5
95     GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
96     GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
97     GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
98
99     // LED6
100    GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
101    GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
102    GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;
103
104    // LED7
105    GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
106    GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
107    GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;
108
109    // LED8
110    GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);

```

```

111 GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
112 GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;
113
114 // LED9
115 GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
116 GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
117 GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;
118
119 // LED10
120 GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
121 GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
122 GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;
123
124 // LED11
125 GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
126 GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
127 GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
128
129 // LED12
130 GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
131 GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
132 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
133
134 // LED13
135 GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
136 GPIO_SetupPinOptions(157, GPIO_OUTPUT, GPIO_PUSHPULL);
137 GpioDataRegs.GPECLEAR.bit.GPIO157 = 1;
138
139 // LED14
140 GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
141 GPIO_SetupPinOptions(158, GPIO_OUTPUT, GPIO_PUSHPULL);
142 GpioDataRegs.GPECLEAR.bit.GPIO158 = 1;
143
144 // LED15
145 GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
146 GPIO_SetupPinOptions(159, GPIO_OUTPUT, GPIO_PUSHPULL);
147 GpioDataRegs.GPECLEAR.bit.GPIO159 = 1;
148
149 // LED16
150 GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
151 GPIO_SetupPinOptions(160, GPIO_OUTPUT, GPIO_PUSHPULL);
152 GpioDataRegs.GPFCLEAR.bit.GPIO160 = 1;
153
154 //WIZNET Reset
155 GPIO_SetupPinMux(0, GPIO_MUX_CPU1, 0);
156 GPIO_SetupPinOptions(0, GPIO_OUTPUT, GPIO_PUSHPULL);
157 GpioDataRegs.GPASET.bit.GPIO0 = 1;
158
159 //ESP8266 Reset
160 GPIO_SetupPinMux(1, GPIO_MUX_CPU1, 0);
161 GPIO_SetupPinOptions(1, GPIO_OUTPUT, GPIO_PUSHPULL);
162 GpioDataRegs.GPASET.bit.GPIO1 = 1;
163
164 //SPIRAM CS Chip Select
165 GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
166 GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSHPULL);
167 GpioDataRegs.GPASET.bit.GPIO19 = 1;
168
169 //DRV8874 #1 DIR Direction
170 GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
171 GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSHPULL);
172 GpioDataRegs.GPASET.bit.GPIO29 = 1;
173
174 //DRV8874 #2 DIR Direction
175 GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
176 GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
177 GpioDataRegs.GPASET.bit.GPIO32 = 1;
178
179 //DAN28027 CS Chip Select
180 GPIO_SetupPinMux(9, GPIO_MUX_CPU1, 0);
181 GPIO_SetupPinOptions(9, GPIO_OUTPUT, GPIO_PUSHPULL);
182 GpioDataRegs.GPASET.bit.GPIO9 = 1;
183
184 //MPU9250 CS Chip Select
185 GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
186 GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL);
187 GpioDataRegs.GPCSET.bit.GPIO66 = 1;
188
189 //WIZNET CS Chip Select
190 GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
191 GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSHPULL);
192 GpioDataRegs.GPDSET.bit.GPIO125 = 1;
193
194 //PushButton 1
195 GPIO_SetupPinMux(4, GPIO_MUX_CPU1, 0);
196 GPIO_SetupPinOptions(4, GPIO_INPUT, GPIO_PULLUP);
197
198 //PushButton 2
199 GPIO_SetupPinMux(5, GPIO_MUX_CPU1, 0);
200 GPIO_SetupPinOptions(5, GPIO_INPUT, GPIO_PULLUP);
201
202 //PushButton 3
203 GPIO_SetupPinMux(6, GPIO_MUX_CPU1, 0);
204 GPIO_SetupPinOptions(6, GPIO_INPUT, GPIO_PULLUP);
205
206 //PushButton 4
207 GPIO_SetupPinMux(7, GPIO_MUX_CPU1, 0);
208 GPIO_SetupPinOptions(7, GPIO_INPUT, GPIO_PULLUP);
209
210 //Joy Stick Pushbutton
211 GPIO_SetupPinMux(8, GPIO_MUX_CPU1, 0);
212 GPIO_SetupPinOptions(8, GPIO_INPUT, GPIO_PULLUP);
213
214 // Clear all interrupts and initialize PIE vector table:
215 // Disable CPU interrupts
216 DINT;
217
218 // Initialize the PIE control registers to their default state.
219 // The default state is all PIE interrupts disabled and flags
220 // are cleared.
221 // This function is found in the F2837xD_PieCtrl.c file.
222 InitPieCtrl();

```

```

223 // Disable CPU interrupts and clear all CPU interrupt flags:
224 IER = 0x0000;
225 IFR = 0x0000;
226
227 // Initialize the PIE vector table with pointers to the shell Interrupt
228 // Service Routines (ISR).
229 // This will populate the entire table, even if the interrupt
230 // is not used in this example. This is useful for debug purposes.
231 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
232 // This function is found in F2837xD_PieVect.c.
233 InitPieVectTable();
234
235 // Interrupts that are used in this example are re-mapped to
236 // ISR functions found within this project
237 EALLOW; // This is needed to write to EALLOW protected registers
238 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
239 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
240 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
241 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
242 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
243 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
244 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
245 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
246 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
247 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
248 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
249
250 PieVectTable.EMIF_ERROR_INT = &SWI_isr;
251 EDIS; // This is needed to disable write to EALLOW protected registers
252
253 // Initialize the CpuTimers Device Peripheral. This function can be
254 // found in F2837xD_CpuTimers.c
255 InitCpuTimers();
256
257 // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
258 // 200MHz CPU Freq, Period (in uSeconds)
259 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);
260 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 20000);
261 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 5000);
262
263 // Enable CpuTimer Interrupt bit TIE
264 CpuTimer0Regs.TCR.all = 0x4000;
265 CpuTimer1Regs.TCR.all = 0x4000;
266 CpuTimer2Regs.TCR.all = 0x4000;
267
268 init_serialSCIA(&SerialA,115200);
269 // init_serialSCIC(&SerialC,115200);
270 // init_serialSCID(&SerialD,115200);
271
272 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
273 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
274 // to CPU-Timer 2: int 12 is for the SWI.
275 IER |= M_INT1;
276 IER |= M_INT8; // SCIC SCID
277 IER |= M_INT9; // SCIA
278 IER |= M_INT12;
279 IER |= M_INT13;
280 IER |= M_INT14;
281
282 // Enable TINT0 in the PIE: Group 1 interrupt 7
283 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
284 // Enable SWI in the PIE: Group 12 interrupt 9
285 PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
286
287 // Enable global Interrupts and higher priority real-time debug events
288 EINT; // Enable Global interrupt INTM
289 ERTM; // Enable Global realtime interrupt DBGM
290
291 // IDLE loop. Just sit and loop forever (optional):
292 while(1)
293 {
294     if (UARTPrint == 1 ) {
295         // Exercise 8
296         ++timeint;
297         time = timeint * 0.25;
298         sinvalue = ampl*sin(2 * PI * frequency * time) + offset;
299
300         satvalue = saturate(sinvalue, SAT_LIMIT);
301         serial_printf(&SerialA,"Timeint = %ld (wrong: %d), Time = %.2fsec, Input = %.3f, SatOut = %.2f\r\n",timeint,timeint, time, sinvalue, satvalue);
302
303         serial_printf(&SerialA,"Num Timer2:%ld Num SerialRX: %ld\r\n",CpuTimer2.InterruptCount,numRXA);
304         UARTPrint = 0;
305     }
306 }
307
308 // float saturate(float input, float saturation_limit){
309 //     if(input > saturation_limit){
310 //         return saturation_limit;
311 //     }
312 //     else if (input < -saturation_limit){
313 //         return -saturation_limit;
314 //     }
315 //     return input;
316 // }
317
318 // SWI_isr, Using this interrupt as a Software started interrupt
319 __interrupt void SWI_isr(void) {
320
321     // These three lines of code allow SWI_isr, to be interrupted by other interrupt functions
322     // making it lower priority than all other Hardware interrupts.
323     PieCtrlRegs.PIEACK.all = PIEACK_GROUP12;
324     asm(" NOP"); // Wait one cycle
325     EINT; // Clear INTM to enable interrupts
326 }

```

```

335
336
337 // Insert SWI ISR Code here.....
338
339
340 numSWIcalls++;
341
342 DINT;
343
344 }
345
346 // cpu_timer0_isr - CPU Timer0 ISR
347 __interrupt void cpu_timer0_isr(void)
348 {
349     CpuTimer0.InterruptCount++;
350
351     numTimer0calls++;
352
353     // if ((numTimer0calls%50) == 0) {
354     //     PieCtrlRegs.PIEIFR12.bit.INTx9 = 1; // Manually cause the interrupt for the SWI
355     // }
356
357     if ((numTimer0calls%250) == 0) {
358         // displayLEDletter(LEDdisplaynum);
359         LEDdisplaynum++;
360         if (LEDdisplaynum == 0xFFFF) { // prevent roll over exception
361             LEDdisplaynum = 0;
362         }
363     }
364
365     // Blink LaunchPad Red LED
366     // GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
367
368     // Acknowledge this interrupt to receive more interrupts from group 1
369     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
370 }
371
372 // cpu_timer1_isr - CPU Timer1 ISR
373 __interrupt void cpu_timer1_isr(void)
374 {
375
376
377     CpuTimer1.InterruptCount++; // MJ Increments the timer's internal interrupt counter
378 }
379
380 uint16_t prev_button_1 = 0;
381 uint16_t prev_button_4 = 0;
382
383 // cpu_timer2_isr CPU Timer2 ISR
384 __interrupt void cpu_timer2_isr(void)
385 {
386
387     // Blink LaunchPad Blue LED
388     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
389
390
391     CpuTimer2.InterruptCount++;
392
393     if ((CpuTimer2.InterruptCount % 50) == 0) {
394         UARTPrint = 1;
395     }
396
397     if((CpuTimer2.InterruptCount % 20) == 0){
398         GpioDataRegs.GPBTOGGLE.bit.GPIO60 = 1; // Toggle LED 11
399         GpioDataRegs.GPATOGGLE.bit.GPIO27 = 1; // Toggle LED 10
400     }
401
402     uint16_t button_1 = GpioDataRegs.GPADAT.bit.GPIO4;
403     uint16_t button_4 = GpioDataRegs.GPADAT.bit.GPIO7;
404
405     if(button_1 == 1 && prev_button_1 == 0){ //If Button 1 is pressed
406         // Toggle LED 12 and 13
407         GpioDataRegs.GPBTOGGLE.bit.GPIO61 = 1; // Toggle LED 12
408         GpioDataRegs.GPETOGGLE.bit.GPIO157 = 1; // Toggle LED 13
409     }
410     prev_button_1 = button_1;
411
412     if(button_4 == 1 && prev_button_4 == 0){ //If Button 4 is pressed
413         // Toggle LED 14 and 15
414         GpioDataRegs.GPETOGGLE.bit.GPIO159 = 1; // Toggle LED 14
415         GpioDataRegs.GPFTOGGLE.bit.GPIO160 = 1; // Toggle LED 15
416     }
417     prev_button_4 = button_4;
418 }

```

```

1  /* SERIAL.C: This code is designed to act as a low-level serial driver for
2     higher-level programming. Ideally, one could simply call init_serial()
3     to initialize the serial port, then use serial_send("data", 4) to send
4     an array of data (8-bit unsigned character strings).
5
6     WRITTEN BY : Paul Miller <pamiller@uiuc.edu>
7     $Id: serial.c,v 1.4 2003/08/08 16:08:56 paul Exp $
8  */
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15
16 #include "F28x_Project.h"    // Device Headerfile and Examples Include File
17 #include <buffer.h>
18 #include <F28379dSerial.h>
19 #include <F2837xD_sci.h>
20
21 serialSCIA_t SerialA;
22 serialSCIB_t SerialB;
23 serialSCIC_t SerialC;
24 serialSCID_t SerialD;
25
26 char RXAdata = 0;
27 char RXBdata = 0;
28 char RXCdata = 0;
29 char RXDdata = 0;
30 uint32_t numRXA = 0;
31 uint32_t numRXB = 0;
32 uint32_t numRXC = 0;
33 uint32_t numRXD = 0;
34
35
36 // for SerialA
37 uint16_t init_serialSCIA(serialSCIA_t *s, uint32_t baud)
38 {
39     volatile struct SCI_REGS *sci;
40     uint32_t clk;
41
42     if (s == &SerialA) {
43         sci = &SciaRegs;
44         s->sci = sci;
45         init_bufferSCIA(&s->TX);
46
47         GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 15);
48         GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_PULLUP);
49         GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 15);
50         GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);
51     } else {
52         return 1;
53     }
54 }
55
56 /* init for standard baud,8N1 comm */
57 sci->SCICTL1.bit.SWRESET = 0;    // init SCI state machines and opt flags
58 sci->SCICCR.all = 0x0;
59 sci->SCICTL1.all = 0x0;
60 sci->SCICTL2.all = 0x0;
61 sci->SCIPRI.all = 0x0;
62 clk = LSPCLK_HZ;                // set baud rate
63 clk /= baud*8;
64 clk--;
65 sci->SCILBAUD.all = clk & 0xFF;
66 sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
67
68 sci->SCICCR.bit.SCICHAR = 0x7;    // (8) 8 bits per character
69 sci->SCICCR.bit.PARITYENA = 0;    // (N) disable parity calculation
70 sci->SCICCR.bit.STOPBITS = 0;    // (1) transmit 1 stop bit
71 sci->SCICCR.bit.LOOPBKENA = 0;    // disable loopback test
72 sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
73
74 sci->SCIFFCT.bit.FFTXDLY = 0;    // TX: zero-delay
75
76 sci->SCIFFTX.bit.SCIFFENA = 1;    // enable SCI fifo enhancements
77 sci->SCIFFTX.bit.TXFIFORESET = 0;
78 sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
79 sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
80 sci->SCIFFTX.bit.TXFFIENA = 0;    // TX: disable fifo interrupt
81 sci->SCIFFTX.bit.TXFIFORESET = 1;
82
83 sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
84 sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag
85 sci->SCIFFRX.bit.RXFFIENA = 1;    // RX: enable fifo interrupt
86 sci->SCIFFRX.bit.RXFFIL = 0x1;    // RX: fifo interrupt

```

```

87     sci->SCIFFRX.bit.RXFIFORESET = 1;    // RX: re-enable fifo
88
89     sci->SCICTL2.bit.RXBKINTENA = 0;      // disable receiver/error interrupt
90     sci->SCICTL2.bit.TXINTENA = 0;        // disable transmitter interrupt
91
92     sci->SCICTL1.bit.TXWAKE = 0;
93     sci->SCICTL1.bit.SLEEP = 0;           // disable sleep mode
94     sci->SCICTL1.bit.RXENA = 1;           // enable SCI receiver
95     sci->SCICTL1.bit.RXERRINTENA = 0;     // disable receive error interrupt
96     sci->SCICTL1.bit.TXENA = 1;           // enable SCI transmitter
97     sci->SCICTL1.bit.SWRESET = 1;         // re-enable SCI
98
99     /* enable PIE interrupts */
100    if (s == &SerialA) {
101        PieCtrlRegs.PIEIER9.bit.INTx1 = 1;
102        PieCtrlRegs.PIEIER9.bit.INTx2 = 1;
103        IER |= (M_INT9);
104        PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
105    }
106
107    return 0;
108 }
109
110 void uninit_serialSCIA(serialSCIA_t *s)
111 {
112     volatile struct SCI_REGS *sci = s->sci;
113
114     /* disable PIE interrupts */
115     if (s == &SerialA) {
116         PieCtrlRegs.PIEIER9.bit.INTx1 = 0;
117         PieCtrlRegs.PIEIER9.bit.INTx2 = 0;
118         IER &= ~M_INT9;
119     }
120     sci->SCICTL1.bit.RXERRINTENA = 0;      // disable receive error interrupt
121     sci->SCICTL2.bit.RXBKINTENA = 0;      // disable receiver/error interrupt
122     sci->SCICTL2.bit.TXINTENA = 0;        // disable transmitter interrupt
123
124     sci->SCICTL1.bit.RXENA = 0;            // disable SCI receiver
125     sci->SCICTL1.bit.TXENA = 0;            // disable SCI transmitter
126 }
127
128
129
130
131 /*****
132  * SERIAL_SEND()
133  *
134  * "User level" function to send data via serial. Return value is the
135  * length of data successfully copied to the TX buffer.
136  *****/
137 uint16_t serial_sendSCIA(serialSCIA_t *s, char *data, uint16_t len)
138 {
139     uint16_t i = 0;
140     if (len && s->TX.size < BUF_SIZESCIA) {
141         for (i = 0; i < len; i++) {
142             if (buf_writeSCIA_1(&s->TX, data[i] & 0x00FF) != 0) break;
143         }
144         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
145         s->sci->SCIFFTX.bit.TXFFIENA = 1;  // TX: enable fifo interrupt
146     }
147     return i;
148 }
149
150 // For SerialB
151 uint16_t init_serialSCIB(serialSCIB_t *s, uint32_t baud)
152 {
153     volatile struct SCI_REGS *sci;
154     uint32_t clk;
155
156     if (s == &SerialB) {
157         sci = &ScibRegs;
158         s->sci = sci;
159         init_bufferSCIB(&s->TX);
160
161         GPIO_SetupPinMux(15, GPIO_MUX_CPU1, 2);
162         GPIO_SetupPinOptions(15, GPIO_INPUT, GPIO_PULLUP);
163         GPIO_SetupPinMux(14, GPIO_MUX_CPU1, 2);
164         GPIO_SetupPinOptions(14, GPIO_OUTPUT, GPIO_PUSHPULL);
165
166     } else {
167         return 1;
168     }
169 }
170
171 /* init for standard baud,8N1 comm */
172 sci->SCICTL1.bit.SWRESET = 0;           // init SCI state machines and opt flags
173 sci->SCICCR.all = 0x0;
174 sci->SCICTL1.all = 0x0;
175 sci->SCICTL2.all = 0x0;

```

```

176     sci->SCIPRI.all = 0x0;
177     clk = LSPCLK_HZ;                      // set baud rate
178     clk /= baud*8;
179     clk--;
180     sci->SCILBAUD.all = clk & 0xFF;
181     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
182
183     sci->SCICCR.bit.SCICHAR = 0x7;         // (8) 8 bits per character
184     sci->SCICCR.bit.PARITYENA = 0;         // (N) disable parity calculation
185     sci->SCICCR.bit.STOPBITS = 0;         // (1) transmit 1 stop bit
186     sci->SCICCR.bit.LOOPBKENA = 0;        // disable loopback test
187     sci->SCICCR.bit.ADDRIDLE_MODE = 0;    // idle-line mode (non-multiprocessor SCI comm)
188
189     sci->SCIFFCT.bit.FFTXDLY = 0;         // TX: zero-delay
190
191     sci->SCIFFTX.bit.SCIFFENA = 1;        // enable SCI fifo enhancements
192     sci->SCIFFTX.bit.TXFIFORESET = 0;
193     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
194     sci->SCIFFTX.bit.TXFFINTCLR = 1;     // TX: clear interrupt flag
195     sci->SCIFFTX.bit.TXFFIENA = 0;       // TX: disable fifo interrupt
196     sci->SCIFFTX.bit.TXFIFORESET = 1;
197
198     sci->SCIFFRX.bit.RXFIFORESET = 0;     // RX: fifo reset
199     sci->SCIFFRX.bit.RXFFINTCLR = 1;      // RX: clear interrupt flag
200     sci->SCIFFRX.bit.RXFFIENA = 1;        // RX: enable fifo interrupt
201     sci->SCIFFRX.bit.RXFFIL = 0x1;       // RX: fifo interrupt
202     sci->SCIFFRX.bit.RXFIFORESET = 1;     // RX: re-enable fifo
203
204     sci->SCICTL2.bit.RXBKINTENA = 0;       // disable receiver/error interrupt
205     sci->SCICTL2.bit.TXINTENA = 0;        // disable transmitter interrupt
206
207     sci->SCICTL1.bit.TXWAKE = 0;
208     sci->SCICTL1.bit.SLEEP = 0;           // disable sleep mode
209     sci->SCICTL1.bit.RXENA = 1;           // enable SCI receiver
210     sci->SCICTL1.bit.RXERRINTENA = 0;     // disable receive error interrupt
211     sci->SCICTL1.bit.TXENA = 1;           // enable SCI transmitter
212     sci->SCICTL1.bit.SWRESET = 1;         // re-enable SCI
213
214     /* enable PIE interrupts */
215     if (s == &SerialB) {
216         PieCtrlRegs.PIEIER9.bit.INTx3 = 1;
217         PieCtrlRegs.PIEIER9.bit.INTx4 = 1;
218         IER |= (M_INT9);
219         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
220     }
221
222     return 0;
223 }
224
225 void uninit_serialSCIB(serialSCIB_t *s)
226 {
227     volatile struct SCI_REGS *sci = s->sci;
228
229     /* disable PIE interrupts */
230     if (s == &SerialB) {
231         PieCtrlRegs.PIEIER9.bit.INTx3 = 0;
232         PieCtrlRegs.PIEIER9.bit.INTx4 = 0;
233         IER &= ~M_INT9;
234     }
235
236     sci->SCICTL1.bit.RXERRINTENA = 0;      // disable receive error interrupt
237     sci->SCICTL2.bit.RXBKINTENA = 0;      // disable receiver/error interrupt
238     sci->SCICTL2.bit.TXINTENA = 0;        // disable transmitter interrupt
239
240     sci->SCICTL1.bit.RXENA = 0;            // disable SCI receiver
241     sci->SCICTL1.bit.TXENA = 0;            // disable SCI transmitter
242 }
243
244
245
246 /*****
247  * SERIAL_SEND()
248  *
249  * "User level" function to send data via serial. Return value is the
250  * length of data successfully copied to the TX buffer.
251  *****/
252 uint16_t serial_sendSCIB(serialSCIB_t *s, char *data, uint16_t len)
253 {
254     uint16_t i = 0;
255     if (len && s->TX.size < BUF_SIZE_SCIB) {
256         for (i = 0; i < len; i++) {
257             if (buf_writeSCIB_1(&s->TX, data[i] & 0x00FF) != 0) break;
258         }
259         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
260         s->sci->SCIFFTX.bit.TXFFIENA = 1;  // TX: enable fifo interrupt
261     }
262     return i;
263 }
264

```



```

265 // for SerialC
266 uint16_t init_serialSCIC(serialSCIC_t *s, uint32_t baud)
267 {
268     volatile struct SCI_REGS *sci;
269     uint32_t clk;
270
271     if (s == &SerialC) {
272         sci = &ScicRegs;
273         s->sci = sci;
274         init_bufferSCIC(&s->TX);
275         GPIO_SetupPinMux(139, GPIO_MUX_CPU1, 6);
276         GPIO_SetupPinOptions(139, GPIO_INPUT, GPIO_PULLUP);
277         GPIO_SetupPinMux(56, GPIO_MUX_CPU1, 6);
278         GPIO_SetupPinOptions(56, GPIO_OUTPUT, GPIO_PUSHPULL);
279
280     } else {
281         return 1;
282     }
283
284     /* init for standard baud, 8N1 comm */
285     sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
286     sci->SCICCR.all = 0x0;
287     sci->SCICTL1.all = 0x0;
288     sci->SCICTL2.all = 0x0;
289     sci->SCIPRI.all = 0x0;
290     clk = LSPCLK_HZ; // set baud rate
291     clk /= baud*8;
292     clk--;
293     sci->SCILBAUD.all = clk & 0xFF;
294     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
295
296     sci->SCICCR.bit.SCICHAR = 0x7; // (8) 8 bits per character
297     sci->SCICCR.bit.PARITYENA = 0; // (N) disable parity calculation
298     sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
299     sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
300     sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
301
302     sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
303
304     sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
305     sci->SCIFFTX.bit.TXFIFORESET = 0;
306     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
307     sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
308     sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
309     sci->SCIFFTX.bit.TXFIFORESET = 1;
310
311     sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
312     sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag
313     sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
314     sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
315     sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
316
317     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
318     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
319
320     sci->SCICTL1.bit.TXWAKE = 0;
321     sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
322     sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
323     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
324     sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
325     sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
326
327     /* enable PIE interrupts */
328     if (s == &SerialC) {
329         PieCtrlRegs.PIEIER8.bit.INTx5 = 1;
330         PieCtrlRegs.PIEIER8.bit.INTx6 = 1;
331         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
332         IER |= (M_INT8);
333     }
334 }
335
336 return 0;
337 }
338
339 void uninit_serialSCIC(serialSCIC_t *s)
340 {
341     volatile struct SCI_REGS *sci = s->sci;
342
343     /* disable PIE interrupts */
344     if (s == &SerialC) {
345         PieCtrlRegs.PIEIER8.bit.INTx5 = 0;
346         PieCtrlRegs.PIEIER8.bit.INTx6 = 0;
347         IER &= ~M_INT8;
348     }
349
350     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
351     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
352     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
353 }

```



```

354     sci->SCICTL1.bit.RXENA = 0;           // disable SCI receiver
355     sci->SCICTL1.bit.TXENA = 0;           // disable SCI transmitter
356 }
357
358
359
360 /*****
361  * SERIAL_SEND()
362  *
363  * "User level" function to send data via serial. Return value is the
364  * length of data successfully copied to the TX buffer.
365  *****/
366 uint16_t serial_sendSCIC(serialSCIC_t *s, char *data, uint16_t len)
367 {
368     uint16_t i = 0;
369     if (len && s->TX.size < BUF_SIZE_SCIC) {
370         for (i = 0; i < len; i++) {
371             if (buf_writeSCIC_1(&s->TX, data[i] & 0x00FF) != 0) break;
372         }
373         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
374         s->sci->SCIFFTX.bit.TXFFIENA = 1;   // TX: enable fifo interrupt
375     }
376     return i;
377 }
378
379 // For Serial D
380 uint16_t init_serialSCID(serialSCID_t *s, uint32_t baud)
381 {
382     volatile struct SCI_REGS *sci;
383     uint32_t clk;
384
385     if (s == &SerialD) {
386         sci = &ScidRegs;
387         s->sci = sci;
388         init_bufferSCID(&s->TX);
389         GPIO_SetupPinMux(105, GPIO_MUX_CPU1, 6);
390         GPIO_SetupPinOptions(105, GPIO_INPUT, GPIO_PULLUP);
391         GPIO_SetupPinMux(104, GPIO_MUX_CPU1, 6);
392         GPIO_SetupPinOptions(104, GPIO_OUTPUT, GPIO_PUSHPULL);
393     }
394     else {
395         return 1;
396     }
397
398     /* init for standard baud, 8N1 comm */
399     sci->SCICTL1.bit.SWRESET = 0;           // init SCI state machines and opt flags
400     sci->SCICCR.all = 0x0;
401     sci->SCICTL1.all = 0x0;
402     sci->SCICTL2.all = 0x0;
403     sci->SCIPRI.all = 0x0;
404     clk = LSPCLK_HZ;                       // set baud rate
405     clk /= baud*8;
406     clk--;
407     sci->SCILBAUD.all = clk & 0xFF;
408     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
409
410     sci->SCICCR.bit.SCICHAR = 0x7;          // (8) 8 bits per character
411     sci->SCICCR.bit.PARITYENA = 0;          // (N) disable parity calculation
412     sci->SCICCR.bit.STOPBITS = 0;          // (1) transmit 1 stop bit
413     sci->SCICCR.bit.LOOPBKENA = 0;          // disable loopback test
414     sci->SCICCR.bit.ADDRIDLE_MODE = 0;      // idle-line mode (non-multiprocessor SCI comm)
415
416     sci->SCIFFCT.bit.FFTXDLY = 0;          // TX: zero-delay
417
418     sci->SCIFFTX.bit.SCIFFENA = 1;          // enable SCI fifo enhancements
419     sci->SCIFFTX.bit.TXFIFORESET = 0;
420     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
421     sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
422     sci->SCIFFTX.bit.TXFFIENA = 0;         // TX: disable fifo interrupt
423     sci->SCIFFTX.bit.TXFIFORESET = 1;
424
425     sci->SCIFFRX.bit.RXFIFORESET = 0;       // RX: fifo reset
426     sci->SCIFFRX.bit.RXFFINTCLR = 1;       // RX: clear interrupt flag
427     sci->SCIFFRX.bit.RXFFIENA = 1;         // RX: enable fifo interrupt
428     sci->SCIFFRX.bit.RXFFIL = 0x1;        // RX: fifo interrupt
429     sci->SCIFFRX.bit.RXFIFORESET = 1;      // RX: re-enable fifo
430
431     sci->SCICTL2.bit.RXBKINTENA = 0;        // disable receiver/error interrupt
432     sci->SCICTL2.bit.TXINTENA = 0;         // disable transmitter interrupt
433
434     sci->SCICTL1.bit.TXWAKE = 0;
435     sci->SCICTL1.bit.SLEEP = 0;            // disable sleep mode
436     sci->SCICTL1.bit.RXENA = 1;            // enable SCI receiver
437     sci->SCICTL1.bit.RXERRINTENA = 0;      // disable receive error interrupt
438     sci->SCICTL1.bit.TXENA = 1;            // enable SCI transmitter
439     sci->SCICTL1.bit.SWRESET = 1;          // re-enable SCI
440
441     /* enable PIE interrupts */
442     if (s == &SerialD) {

```

```

443     PieCtrlRegs.PIEIER8.bit.INTx7 = 1;
444     PieCtrlRegs.PIEIER8.bit.INTx8 = 1;
445     PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
446     IER |= (M_INT8);
447 }
448
449     return 0;
450 }
451
452 void uninit_serialSCID(serialSCID_t *s)
453 {
454     volatile struct SCI_REGS *sci = s->sci;
455
456     /* disable PIE interrupts */
457     if (s == &SerialD) {
458         PieCtrlRegs.PIEIER8.bit.INTx7 = 0;
459         PieCtrlRegs.PIEIER8.bit.INTx8 = 0;
460         IER &= ~M_INT8;
461     }
462
463     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
464     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
465     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
466
467     sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
468     sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
469 }
470
471
472
473 /*****
474  * SERIAL_SEND()
475  *
476  * "User level" function to send data via serial. Return value is the
477  * length of data successfully copied to the TX buffer.
478  *****/
479 uint16_t serial_sendSCID(serialSCID_t *s, char *data, uint16_t len)
480 {
481     uint16_t i = 0;
482     if (len && s->TX.size < BUF_SIZE_SCID) {
483         for (i = 0; i < len; i++) {
484             if (buf_writeSCID_1(&s->TX, data[i] & 0x00FF) != 0) break;
485         }
486         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
487         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
488     }
489     return i;
490 }
491
492
493
494 /*****
495  * TXxINT_DATA_SENT()
496  *
497  * Executed when transmission is ready for additional data. These functions
498  * read the next char of data and put it in the TXBUF register for transfer.
499  *****/
500 #ifdef _FLASH
501 #pragma CODE_SECTION(TXAINT_data_sent, ".TI.ramfunc");
502 #endif
503 __interrupt void TXAINT_data_sent(void)
504 {
505     char data;
506     if (buf_readSCIA_1(&SerialA.TX, 0, &data) == 0) {
507         while ( (buf_readSCIA_1(&SerialA.TX, 0, &data) == 0)
508             && (SerialA.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
509             buf_removeSCIA(&SerialA.TX, 1);
510             SerialA.sci->SCITXBUF.all = data;
511         }
512     } else {
513         SerialA.sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
514     }
515     SerialA.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
516     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
517 }
518
519
520 //for serialB
521 #ifdef _FLASH
522 #pragma CODE_SECTION(TXBINT_data_sent, ".TI.ramfunc");
523 #endif
524 __interrupt void TXBINT_data_sent(void)
525 {
526     char data;
527     if (buf_readSCIB_1(&SerialB.TX, 0, &data) == 0) {
528         while ( (buf_readSCIB_1(&SerialB.TX, 0, &data) == 0)
529             && (SerialB.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
530             buf_removeSCIB(&SerialB.TX, 1);
531             SerialB.sci->SCITXBUF.all = data;

```

```

532     }
533     } else {
534         SerialB.sci->SCIFFTX.bit.TXFFIENA = 0;        // TX: disable fifo interrupt
535     }
536     SerialB.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
537     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
538 }
539
540
541 //for serialC
542 #ifdef _FLASH
543 #pragma CODE_SECTION(TXCINT_data_sent, ".TI.ramfunc");
544 #endif
545 __interrupt void TXCINT_data_sent(void)
546 {
547     char data;
548     if (buf_readSCIC_1(&SerialC.TX,0,&data) == 0) {
549         while ( (buf_readSCIC_1(&SerialC.TX,0,&data) == 0)
550             && (SerialC.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
551             buf_removeSCIC(&SerialC.TX, 1);
552             SerialC.sci->SCITXBUF.all = data;
553         }
554     } else {
555         SerialC.sci->SCIFFTX.bit.TXFFIENA = 0;        // TX: disable fifo interrupt
556     }
557     SerialC.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
558     PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
559 }
560
561 //for serialD
562 #ifdef _FLASH
563 #pragma CODE_SECTION(TXDINT_data_sent, ".TI.ramfunc");
564 #endif
565 __interrupt void TXDINT_data_sent(void)
566 {
567     char data;
568     if (buf_readSCID_1(&SerialD.TX,0,&data) == 0) {
569         while ( (buf_readSCID_1(&SerialD.TX,0,&data) == 0)
570             && (SerialD.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
571             buf_removeSCID(&SerialD.TX, 1);
572             SerialD.sci->SCITXBUF.all = data;
573         }
574     } else {
575         SerialD.sci->SCIFFTX.bit.TXFFIENA = 0;        // TX: disable fifo interrupt
576     }
577     SerialD.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
578     PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
579 }
580
581 //for SerialA
582 #ifdef _FLASH
583 #pragma CODE_SECTION(RXAINT_recv_ready, ".TI.ramfunc");
584 #endif
585 __interrupt void RXAINT_recv_ready(void)
586 {
587     RXAdata = SciaRegs.SCIRXBUF.all;
588
589     /* SCI PE or FE error */
590     if (RXAdata & 0xC000) {
591         SciaRegs.SCICTL1.bit.SWRESET = 0;
592         SciaRegs.SCICTL1.bit.SWRESET = 1;
593         SciaRegs.SCIFFRX.bit.RXFIFORESET = 0;
594         SciaRegs.SCIFFRX.bit.RXFIFORESET = 1;
595     } else {
596         RXAdata = RXAdata & 0x00FF;
597
598         numRXA ++;
599
600         if(RXAdata == 'a'){
601             GpioDataRegs.GPASET.bit.GPI034 = 1;
602         }
603         else if(RXAdata == 'b'){
604             GpioDataRegs.GPBCLEAR.bit.GPI034 = 1;
605         }
606     }
607
608     SciaRegs.SCIFFRX.bit.RXFFINTCLR = 1;
609     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
610 }
611 }
612
613 //for SerialB
614 #ifdef _FLASH
615 #pragma CODE_SECTION(RXBINT_recv_ready, ".TI.ramfunc");
616 #endif
617 __interrupt void RXBINT_recv_ready(void)
618 {
619     RXBdata = ScibRegs.SCIRXBUF.all;
620

```

```

621  /* SCI PE or FE error */
622  if (RXBdata & 0xC000) {
623      ScibRegs.SCICTL1.bit.SWRESET = 0;
624      ScibRegs.SCICTL1.bit.SWRESET = 1;
625      ScibRegs.SCIFFRX.bit.RXFIFORESET = 0;
626      ScibRegs.SCIFFRX.bit.RXFIFORESET = 1;
627  } else {
628      RXBdata = RXBdata & 0x00FF;
629      // Do something with recieved character
630      numRXB ++;
631  }
632
633  ScibRegs.SCIFFRX.bit.RXFFINTCLR = 1;
634  PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
635 }
636
637
638 // for SerialC
639 #ifndef _FLASH
640 #pragma CODE_SECTION(RXCINT_recv_ready, ".TI.ramfunc");
641 #endif
642 __interrupt void RXCINT_recv_ready(void)
643 {
644     RXCdata = ScicRegs.SCIRXBUF.all;
645
646     /* SCI PE or FE error */
647     if (RXCdata & 0xC000) {
648         ScicRegs.SCICTL1.bit.SWRESET = 0;
649         ScicRegs.SCICTL1.bit.SWRESET = 1;
650         ScicRegs.SCIFFRX.bit.RXFIFORESET = 0;
651         ScicRegs.SCIFFRX.bit.RXFIFORESET = 1;
652     } else {
653         RXCdata = RXCdata & 0x00FF;
654         numRXC ++;
655     }
656 }
657
658 ScicRegs.SCIFFRX.bit.RXFFINTCLR = 1;
659 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
660 }
661
662
663 //for SerialD
664 #ifndef _FLASH
665 #pragma CODE_SECTION(RXDINT_recv_ready, ".TI.ramfunc");
666 #endif
667 __interrupt void RXDINT_recv_ready(void)
668 {
669     RXDdata = ScidRegs.SCIRXBUF.all;
670
671     /* SCI PE or FE error */
672     if (RXDdata & 0xC000) {
673         ScidRegs.SCICTL1.bit.SWRESET = 0;
674         ScidRegs.SCICTL1.bit.SWRESET = 1;
675         ScidRegs.SCIFFRX.bit.RXFIFORESET = 0;
676         ScidRegs.SCIFFRX.bit.RXFIFORESET = 1;
677     } else {
678         RXDdata = RXDdata & 0x00FF;
679         numRXD ++;
680     }
681 }
682
683 ScidRegs.SCIFFRX.bit.RXFFINTCLR = 1;
684 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
685 }
686
687
688 // SerialA only setup for Tera Term connection
689 char serial_printf_bufSCIA[BUF_SIZE_SCIA];
690
691 uint16_t serial_printf(serialSCIA_t *s, char *fmt, ...)
692 {
693     va_list ap;
694
695     va_start(ap, fmt);
696     vsprintf(serial_printf_bufSCIA, fmt, ap);
697     va_end(ap);
698
699     return serial_sendSCIA(s, serial_printf_bufSCIA, strlen(serial_printf_bufSCIA));
700 }

```